Data Mining 2

Topic 02 : Feature Engineering

Lecture 03 : Introduction to Feature Engineering

Dr Kieran Murphy

Department of Computing and Mathematics, Waterford Institute of Technology.
(Kieran.Murphy@setu.ie)

Spring Semester, 2024

Outline

- Need for feature engineering
- Generic feature engineering steps

Numeric Feature I

Numeric data typically represents data in the form of scalar values depicting

- observations, recordings or measurements
- frequencies or counts

It is very rich data but the pattern of interest may often be lost within the noise. Common feature engineering techniques are

- **Binarisation** — Generate a binary feature from a numeric feature.
- **Rounding** — reduce the precision in the data to simplify decision tree models, or as a step towards representing as categorical.
- **Binning** —- Divide numerical scale into bins (think histogram) to then convert to a categorical feature.
  - Bins can be fixed (equal) width or adaptive.
  - Bin width / number of bins can be determined using decision trees.

Example —- Binarisation

Consider building a recommendation system for song recommendations:

- Want to know if a person is interested or has listened to a particular song.
- If given a dataset with a feature `listen_count`, storing the number of time each individual has listened to each song, then we need to binarise our `listen_count` field as follows:

```
tmp = np.array(df["listen_count"])
tmp[tmp >= 1] = 1
df["listened"] = tmp
```

or (using more modern pandas)

```
df["listened"] = (df["listen_count"]>0).astype(int)
```

or (using sklearn)

```
from sklearn.preprocessing import Binarizer
bn = Binarizer(threshold=0.9)
tmp = bn.transform([df["listen_count"]])[0]
df["listened"] = tmp
```

Categorical Feature

Dealing with special categories

Imagine you have a categorical attribute, like `Item_Color` that can be 'Red', 'Blue' or 'Unknown'. All three categories are treated equally. But there are situations in which the 'unknown' should be treated separately — for example, when it arises from missing values in the original data and the instances of missing values is linked to the target.
In this case we need to create an additional binary feature to indicate the special category.

Merging Rare (infrequent) categories

See titles in Titanic dataset.

Encodings

- **Label Encoding** — label levels as $1, 2, \ldots$ based on order seen.
- **Ordinal Encoding** — label levels as $1, 2, \ldots$ based on natural order.
- **One-Hot Encoding** — separate binary indicator variable for each level.
- **Frequency Encoding** — level values are replaced by the frequency that the level appeared.
- **(K-Fold) Target Encoding** — level values are replaced by target statistic (mean) for that level.
- $\ldots$ many more $\ldots$

Frequency Encoding

In frequency encoding (count encoding), replace each level/category with the count of how often it appears in the dataset.

	Temperature	Color	Target	FreqEnc_Temperature
0	Hot	Red	1	0.4
1	Cold	Yellow	1	0.2
2	Very Hot	Blue	1	0.1
3	Warm	Blue	0	0.3
4	Hot	Red	1	0.4
5	Warm	Yellow	0	0.3
6	Warm	Red	1	0.3
7	Hot	Yellow	0	0.4
8	Hot	Yellow	1	0.4
9	Cold	Yellow	1	0.2

	Temperature	Color	Target	FreqEnc_Color
0	Hot	Red	1	0.3
1	Cold	Yellow	1	0.5
2	Very Hot	Blue	1	0.2
3	Warm	Blue	0	0.2
4	Hot	Red	1	0.3
5	Warm	Yellow	0	0.5
6	Warm	Red	1	0.3
7	Hot	Yellow	0	0.5
8	Hot	Yellow	1	0.5
9	Cold	Yellow	1	0.5

Frequency Encoding II

- ✔ **Effective with High Cardinality** — Since each category is replaced by its frequency, the encoding is not affected by number of categories, i.e., does not increase dimensionality (in contrast with One-Hot Encoding).
- ✔ **Preservation of Information** — The frequency of each category is preserved. This can be useful, e.g., in a sales dataset, the frequency of each product can be an indicator of the product's popularity.
- ✘ **Doesn't Handle Unseen Categories** — Levels/categories that do not appear in the train dataset are not encoded (worse than One-Hot encoding)
- ✘ **Loss of Unique Categories** — If two categories have the same frequency, they will be represented by the same value after frequency encoding.
- ✔ **Simplicity** — Calculation is trivial and not computationally expensive.
- ✔ **Works with all Models** — Suitable for both tree-based and non-tree-based models. (In contrast Ordinal Encoding does not work well with linear models).
- ✘ **Risk of Overfitting for Rare Categories** — Low frequency might not be a good representation of importance, leading to overfitting, where the model learns to rely too much on infrequent categories.
- ✘ **Not Suitable for Ordinal Variables** — Natural order is lost in frequency encoding.

Target Encoding

I

In target encoding replaces each level/category with the average value of the target variable for that category.

Rec-No	Temperature	Color	Target
1	Hot	Red	1
2	Cold	Yellow	1
3	Very Hot	Blue	1
4	Warm	Blue	0
5	Hot	Red	1
6	Warm	Yellow	0
7	Warm	Red	1
8	Hot	Yellow	0
9	Hot	Yellow	1
10	Cold	Yellow	1

Step-1	Sum of Target
Cold	2
Hot	3
Very Hot	1
Warm	1

Step-2	Count of Target
Cold	2
Hot	4
Very Hot	1
Warm	3

Step-3	Mean
Cold	1.00
Hot	0.75
Very Hot	1.00
Warm	0.33

Rec-No	Temperature	Color	Target
1	0.75	Red	1
2	1.00	Yellow	1
3	1.00	Blue	1
4	0.33	Blue	0
5	0.75	Red	1
6	0.33	Yellow	0
7	0.33	Red	1
8	0.75	Yellow	0
9	0.75	Yellow	1
10	1.00	Yellow	1

https://towardsdatascience.com/all-about-categorical-variable-encoding-305f3361fd02

Target Encoding II

Target encoding has similar advantages and disadvantages as feature encoding. However target encoding has a fix for rare or unseen categories, based on smoothing.

$$\text{encoding} = \text{weight} \times \text{in_category} + (1 - \text{weight}) \times \text{overall}$$

where

- weight is a value between 0 and 1 calculated from the category frequency.
- in_category is the target mean value for rows with the given category value.
- in_category is the overall target mean value.

Then we compute weight using

$$\text{weight} = \frac{n}{n+m}$$

where

- n is the number of times that the category occurs in the data.
- m is the "smoothing factor".
  - Setting $m = 0$, means that weight $= 1$ so above formula reduces to

    $$\text{encoding} = \text{in_category}$$

    and no smoothing takes place.
  - Larger values of m put more weight on the overall estimate and less on the in_category estimate.

Date/Time Features

- Decomposing date into day, month, year, day of week
- Decomposing time in hour, minute, second.
- Subtracting dates to get age.
- Extracting day of week
- Categorising into workday/weekend — see Kaggle, Bike hire dataset
- Duration between two times — e.g., harmonic transformation

```
def make_harmonic_features(value, period=24):
    value *= 2 * np.pi / period
    return np.cos(value), np.sin(value)
```

This transformation preserves the distance between points, which is important for algorithms that estimate distance (kNN, SVM, k-means ...)

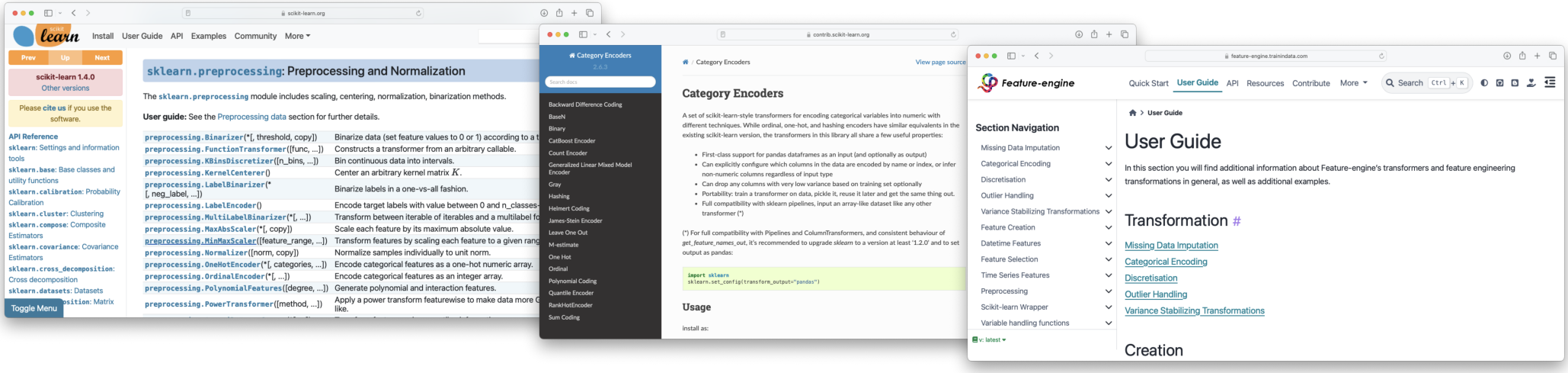
Libraries for Category Encoding

scikit-learn

- Most commom encoders and consistent style: `fit`, `transform`, and `fit_transform` methods.

Python module `category_encoders`

- extra scikit-learn-style transformers for encoding.

Python module `feature_engine` (via condo-forge)

- Only recently started using this, but it is comprehensive and compatable with scikit-learn.